Gradient Descent: Algorithms, Mathematics, and Visualizations

Comprehensive Guide Covering Variants, Math, and Pitfalls

This document provides a detailed exploration of gradient descent optimization, including mathematics, types (batch, stochastic, mini-batch), challenges, and illustrative diagrams.

Autor: Generated for Maru by Grok

February 2026

1. Introduction to Gradient Descent

Gradient Descent (GD) is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. It is widely used in machine learning for minimizing loss functions. The algorithm iteratively updates parameters in the direction opposite to the gradient of the function, proportional to the negative gradient.

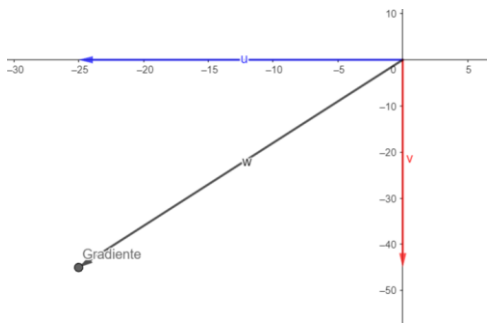

Figure 1: Visualization of gradient vectors in a 2D parameter space (W, V), showing the direction of steepest descent

The core idea is to take steps proportional to the **negative gradient** of the cost function with respect to parameters. This moves towards the minimum cost.

GD is foundational for training neural networks and other models.

2. Mathematical Foundations

The gradient of a function **J(θ)** is a vector of partial derivatives: **∇J(θ) = [∂J/∂θ₁, ..., ∂J/∂θₙ]**. It points in the direction of steepest ascent.

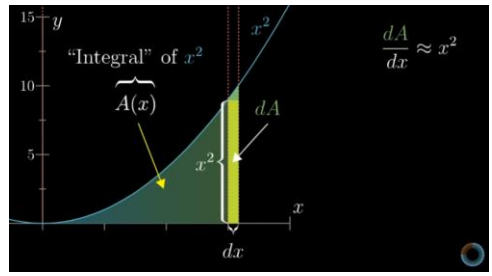

Figure 2: Geometric interpretation of the derivative as slope of 'integral' of x^2

$$\nabla J(\theta) = \left[\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_n}\right]$$

Equation 1: Gradient vector definition

The parameter update rule is:

θ ← θ - α ∇J(θ)

where **α** is the learning rate controlling step size.

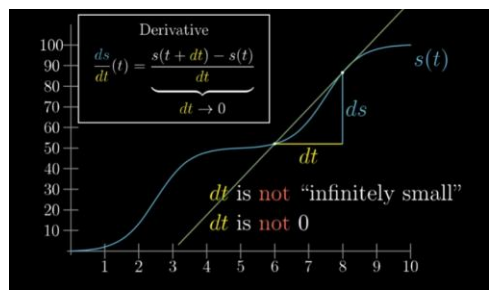

Figure 3: Finite difference approximation of the derivative

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)})$$

Equation 2: Gradient Descent update rule

3. Types of Gradient Descent

- **Batch Gradient Descent (BGD)**: Computes gradient over entire dataset. Precise but slow for large data.
- **Stochastic Gradient Descent (SGD)**: Updates using one sample at a time. Fast, noisy path to minimum.

- **Mini-batch Gradient Descent**: Compromise using small batches. Balances speed and stability.

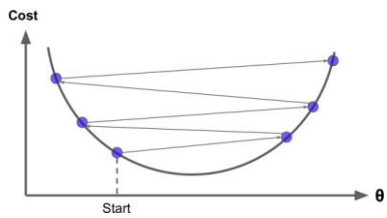

Figure 4: Contour plot showing paths for different GD variants

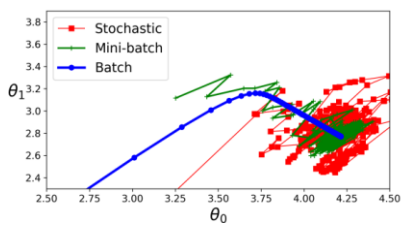

Figure 5: Cost vs. epochs comparison for Stochastic (red), Mini-batch (green), Batch (blue)

Table 1: Comparison of Gradient Descent Variants

Type	Dataset Usage	Update Frequency	Pros	Cons
Batch GD	Full dataset	Once per epoch	Smooth convergence, exact direction	Computationally expensive, slow for large data
Stochastic GD	1 sample	Every sample	Fast updates, escapes local minima	Noisy, erratic convergence
Mini-batch GD	Small batch (e.g., 32-256)	Every mini-batch	Faster than batch, less noisy than SGD	Batch size tuning required

4. Learning Rate and Challenges

Choosing **α** is crucial. Too large: overshoot minimum. Too small: slow convergence.

Common pitfalls include local minima, plateaus, saddle points, and cliffs.

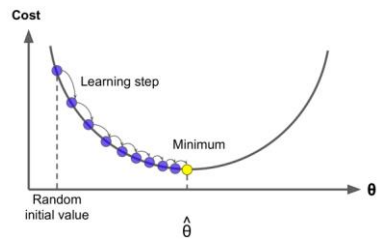

Figure 6: Illustration of learning step size on a parabolic cost function

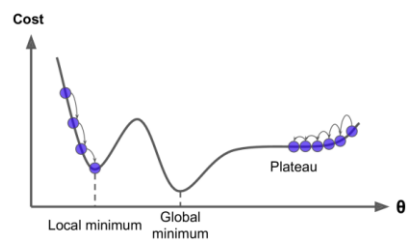

Figure 7: Pitfalls in Gradient Descent: local/global minima, plateau

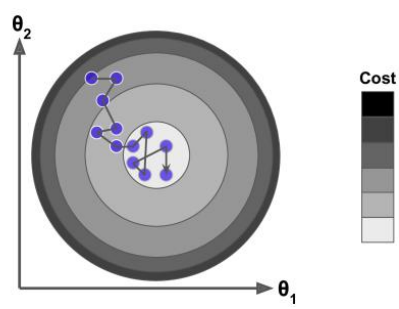

Figure 8: SGD trajectory on a cost contour

Solutions: Momentum, Adam optimizer, learning rate scheduling, line search.

5. Multidimensional and Advanced Visualizations

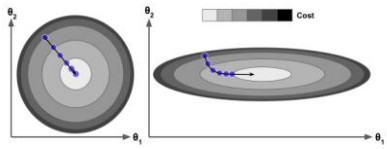

Figure 9: Gradient descent in 2D feature space

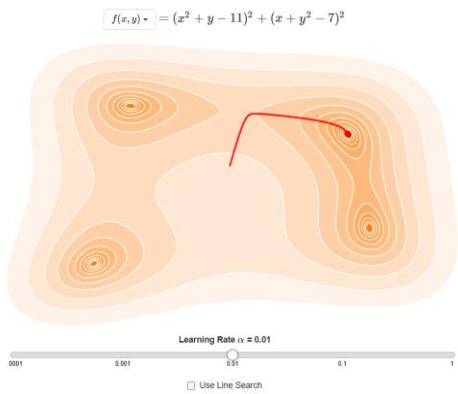

Figure 10: Contour plot with learning rate effects

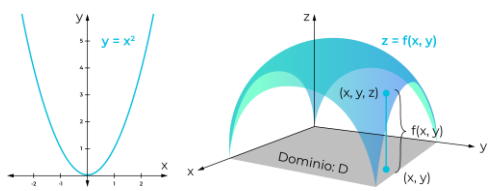

Figure 11: Line search in optimization

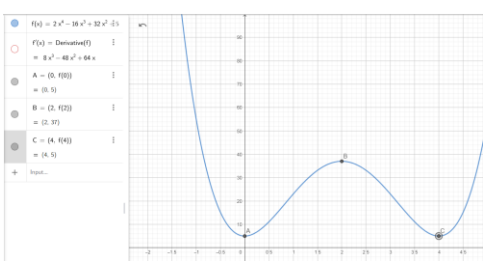

Figure 12: Cost function with multiple minima

In higher dimensions, GD navigates complex surfaces. Visualizations aid understanding the algorithm's behavior.

Domain considerations ensure differentiability over the parameter space.

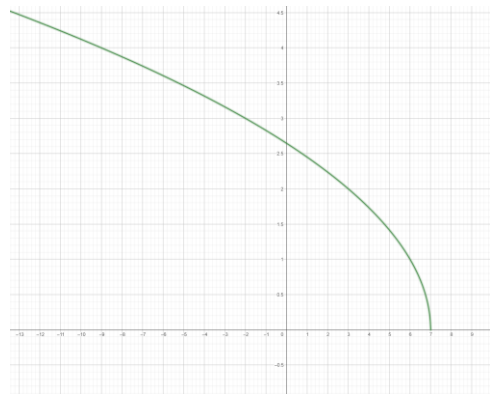

Figure 13: Function domain and gradient surface

Conclusion

Gradient Descent remains a cornerstone of optimization despite limitations. Modern variants like Adam address many issues, enabling training of large-scale models.

For implementation, monitor cost curves and adjust hyperparameters accordingly.